StoryLand

AI-Powered Storytelling & Reading Platform

MERN Stack Edition (MongoDB, Express.js, React, Node.js)

1. Project Overview

StoryLand is an educational web application designed to encourage children to read through engaging, age-appropriate stories generated using artificial intelligence.

Teachers often need creative reading materials adapted to their students' ages, interests, and language levels. Preparing these materials can be time-consuming, especially when working with different classes.

StoryLand helps teachers generate original stories using AI, organize their students into classes, and assign reading activities. Students can access assigned stories through a playful, child-friendly interface, read them page by page, and track their progress.

The platform supports story generation in Arabic, French, and English. Teachers select a theme, target age, language, and story length before generating content. Every generated story must be reviewed and approved by a teacher before students can read it.

The main objectives are:

- **Encourage reading:** Provide children with an enjoyable and accessible digital reading experience.
- **Simplify content creation:** Help teachers generate original stories adapted to their students.
- **Organize classroom reading:** Allow teachers to manage classes and assign stories.
- **Track reading progress:** Enable students to monitor their own progress while teachers follow reading completion across their classes.
- **Support responsible AI use:** Ensure stories are age-appropriate and reviewed before publication.

The application should focus on storytelling, reading, and progress tracking rather than becoming a complete learning management system.

2. Users and Roles

StoryLand supports two user roles.

Teacher

Teachers manage their own classes, create stories, and monitor student reading activities. They should be able to:

- Register and log in.
- Create and manage classes.
- Add students and manage their accounts.
- Generate stories using AI.
- Review, edit, and publish generated stories.
- Manage their story library.
- Assign stories to one or more classes.
- View individual student reading progress.

- Monitor reading completion for assigned stories.

Each class belongs to one teacher. Teachers should only access their own classes, students, stories, and related progress information.

Student

Students use StoryLand to read stories assigned by their teacher. They should be able to:

- Log in using an account created by their teacher.
- View stories assigned to their class.
- Read stories page by page.
- Explicitly mark pages as completed.
- Resume reading from the last opened page.
- View their own reading progress and completion status.
- Revisit previously completed stories.

Students cannot create or manage stories, classes, or other student accounts. They must not access other students' private information.

3. Functional Requirements

3.1 Must Have: Required Features

A. Authentication and Account Management

The platform must provide secure authentication with role-based access control.

Teachers should be able to register and manage their accounts. Students should log in using accounts created by their teacher.

Each user should have access to an appropriate interface based on their role.

The backend must enforce data ownership and access restrictions, ensuring that teachers can only manage their own classes and students, while students can only access their assigned stories and personal progress.

B. Class and Student Management

Teachers must be able to create classes, edit class information, and add or remove students. Each class should contain a name, optional description, and its associated students.

Teachers should be able to view their classes, access student lists, and review assigned stories.

A student belongs to a teacher-managed class, and the application must handle student removal or class deletion without leaving inconsistent records.

Complex school administration, multi-teacher classes, and advanced permission management are not required.

C. AI Story Generation

Teachers must be able to generate original stories using AI. The generation form should include:

Parameter	Description
Theme or Topic	Friendship, adventure, animals, nature, kindness, etc.
Target Age	Intended age or age group
Language	Arabic, French, or English
Number of Pages	5, 8, 10, 13, or 15

The AI must produce a coherent, age-appropriate story containing exactly the selected number of pages.

Each story should have a title, a logical sequence of events, and a meaningful ending. Language complexity and content should be appropriate for the target age.

The generated result must be structured so the application can validate its content and page count. If generation fails or produces an invalid result, the application must provide an appropriate error instead of saving an incomplete story as valid.

D. Story Management and Publication

Teachers should be able to review generated stories, edit their titles and page content, and save them to a personal story library. Stories should support two statuses:

Status	Description
Draft	The story is being reviewed and is not available to students.
Published	The teacher has approved the story, and it can be assigned to classes.

Every AI-generated story must be reviewed and approved by the teacher before publication. Teachers should be able to view and manage their previously created stories.

Once students have started reading an assigned story, the application must prevent structural changes that would invalidate their progress, such as deleting or reordering pages.

A complex story-versioning system is not required.

E. Story Assignment

Teachers must be able to assign published stories to one or more of their classes. A story should be created once and reused across multiple classes without duplicating its content. Students should automatically see stories assigned to their class.

The platform must ensure that:

- Only published stories can be assigned.
- Teachers can only assign stories to their own classes.
- Students can only access stories assigned to their class.
- Each student's progress remains independent, even when several classes read the same story.

F. Student Reading Experience

Students should access assigned stories through a colorful, playful, and child-friendly reading interface. Stories must be displayed page by page, with simple navigation and clear page numbers.

The interface should support appropriate text direction, including right-to-left reading for Arabic.

Students should be able to navigate between pages and explicitly mark individual pages as completed.

A gamified visual experience is expected as part of the UI/UX design, but the application does not require points, badges, reward systems, or complex gamification logic.

G. Reading Progress and Automatic Resume

The platform must track each student's reading progress separately for every assigned story. Two pieces of information must be recorded independently:

- **Completed pages:** Pages explicitly marked as completed by the student.
- **Last opened page:** The most recently opened page, used to resume reading.

Opening a page must not automatically mark it as completed. A story becomes Completed only when all its pages have been explicitly completed.

For example, if a student has completed five pages of a ten-page story and is currently viewing page six, the progress is 50%. When the student returns, the application should automatically open page six.

Revisiting completed pages must not reset progress or count them twice.

H. Student Reading Dashboard

Students must have a personal dashboard displaying their assigned stories and reading progress. For each story, the dashboard should show:

- Story title.
- Completed and total pages.
- Completion percentage.
- Reading status.
- An action to start or continue reading.

The supported reading statuses are:

Status	Description
Not Started	The student has not opened the story.
In Progress	The student has started but has not completed all pages.
Completed	All pages have been explicitly completed.

Students should be able to reopen completed stories without losing their progress.

I. Teacher Reading Dashboard

Teachers must be able to monitor reading progress for students in their classes. The dashboard should allow teachers to select a class and assigned story, then view:

- Students assigned to the story.
- Number of completed pages per student.
- Reading completion percentage.

- Reading status for each student.
- Students who have not started, are in progress, or have completed the story.

For example:

Student	Progress	Status
Sara	10/10 pages	Completed
Adam	5/10 pages	In Progress
Lina	2/10 pages	In Progress
Youssef	0/10 pages	Not Started

All information must come from actual reading records. Advanced reading analytics or automated student performance assessments are not required.

3.2 Should Have: Recommended Features

These features improve the platform without being essential to its core functionality.

Story Search and Filtering

Teachers should be able to search and filter stories by title, language, age group, or publication status.

Story Categories

Teachers should be able to organize stories into simple categories such as Adventure, Friendship, Animals, or Nature.

Additional Language Translation

Teachers should be able to translate an existing story into another supported language using AI. Translated stories should preserve the main narrative and remain age-appropriate. Teachers must review translated content before publishing it.

Complex multilingual version management and synchronization are not required.

Class Reading Summaries

Teachers should be able to view basic summaries showing how many students have started, are reading, or have completed an assigned story.

3.3 Could Have: Optional Features

Downloadable Stories

Allow teachers or students to download stories in a readable format for offline viewing.

Additional features should remain relevant to storytelling and reading without unnecessarily expanding the project's scope.

4. AI Integration

4.1 Purpose

The main purpose of AI in StoryLand is to help teachers generate original, age-appropriate reading materials efficiently. AI should transform a teacher's selected theme, language, target age, and page count into a coherent story that can be reviewed, edited, and published.

AI supports content creation but does not replace teacher judgment or approval.

4.2 Expected AI Workflow

The AI integration should follow this workflow:

- **1. Story Configuration:** The teacher selects the theme, target age, language, and number of pages.
- **2. Generation Request:** The backend validates the input and sends it to the selected AI model or service.
- **3. Story Generation:** AI produces an original story with a title and the requested number of ordered pages.
- **4. Response Validation:** The backend validates the structure, required fields, and page count.
- **5. Content Safety:** The application applies appropriate safeguards against unsuitable content.
- **6. Teacher Review:** The teacher reviews and edits the generated story.
- **7. Publication:** The approved story becomes available for class assignment.

AI-generated content must remain unpublished until the teacher approves it.

4.3 Expected AI Capabilities

The AI should be capable of:

- Generating original stories from a theme or topic.
- Adapting vocabulary and content to the selected age.
- Supporting Arabic, French, and English.
- Producing exactly the requested number of pages.
- Maintaining narrative consistency between pages.
- Generating a meaningful beginning, development, and ending.
- Returning structured output for backend validation.
- Avoiding inappropriate content for young readers.

For example, a teacher selects: Theme: Friendship and teamwork; Target Age: 8 years old; Language: French; Pages: 8. The AI generates an eight-page French story about children learning to cooperate to solve a problem. The teacher reviews the content, makes any necessary changes, and publishes it.

4.4 Safety and Reliability

StoryLand must prioritize child safety and reliable AI output.

- Generated content must be suitable for the selected age group.
- Stories must avoid explicit, graphic, hateful, or otherwise inappropriate content for children.

- The application should use available model safety controls or appropriate content checks.
- Teachers must review and approve stories before publication.
- Invalid or incomplete AI responses must not be published.
- The system must handle AI failures without affecting existing stories or reading progress.
- Generated content should be stored in a structured format that the backend can validate.

The platform should not assume that AI-generated content is automatically safe or accurate.

4.5 Technical Approach

AI integration must be handled directly through the Express.js backend using an appropriate AI API or model.

Students may choose suitable AI models, prompting approaches, and response-validation strategies (for example, JSON output validated on the backend with a schema validation library).

Training a custom AI model is not required. The solution should be achievable using free APIs, open-source models, or suitable free-tier services.

5. Technology Stack and Deployment

5.1 Required Technology Stack (MERN)

The project must be built exclusively with the MERN stack. No other framework or database is required or expected.

Component	Technology
Database	MongoDB (with Mongoose as the ODM)
Backend	Node.js with Express.js
Frontend	React.js (functional components and hooks, with React Router for navigation)
AI Integration	AI API or model integrated directly through Express.js
Version Control	Git and GitHub

MongoDB is well suited to StoryLand, particularly for storing structured stories with multiple pages (for example, a story document containing an ordered array of pages). Related records such as teachers, students, classes, assignments, and reading progress can be stored in separate collections linked by references.

No specific database schema or data model is prescribed.

The Express.js backend remains responsible for authentication, authorization, class management, story management, reading progress, and AI integration. The React frontend consumes the backend REST API.

5.2 Deployment

The completed application must be deployed and accessible online. The frontend, backend, database, and AI integration must function correctly in the deployed environment.

Suggested services include:

Component	Suggested Provider
React Frontend	Vercel or Netlify
Express.js Backend	Railway or Render
MongoDB	MongoDB Atlas

These services are suggestions, not mandatory choices. Students may use suitable alternatives, preferably with available free-tier resources.

6. Expected Deliverables

The final submission should include:

- A functional full-stack StoryLand application built with the MERN stack.
- Secure Teacher and Student authentication.
- Class and student management.
- AI-powered story generation in Arabic, French, or English.
- Teacher review, editing, and publication of stories.
- A story library and class assignment functionality.
- A child-friendly, page-by-page reading interface.
- Explicit page completion tracking and automatic reading resume.
- Student and teacher reading progress dashboards.
- Appropriate ownership and access restrictions.
- Persistent database integration using MongoDB.
- AI response validation and error handling.
- A GitHub repository containing the source code.
- A README with installation, configuration, and deployment instructions.
- Sample classes, student accounts, and stories for demonstration.
- A working online deployment.
- A final presentation demonstrating the application's main workflows.

Example End-to-End Scenario

- **1. Teacher Registration & Class Setup:** A teacher creates a StoryLand account, logs in, and creates a class named Young Readers for children aged eight.
- **2. Student Management:** The teacher adds five students to the class, creating individual accounts that allow each student to access their assigned reading activities.
- **3. AI Story Generation:** The teacher opens the story generator and selects the theme Friendship and Teamwork, a target age of eight, English as the language, and eight pages. The AI generates

an original story titled The Little Explorer, divided into eight ordered pages with age-appropriate content.

- **4. Story Review & Publication:** The teacher reviews the generated story, corrects some wording, and verifies that the content is suitable for the students. Once satisfied, the teacher approves and publishes the story.
- **5. Story Assignment:** The teacher assigns The Little Explorer to the Young Readers class. The story automatically becomes available in each student's reading library.
- **6. Student Reading Experience:** A student named Adam logs in, opens the assigned story, and begins reading through the colorful, child-friendly interface. He reads the first four pages, explicitly marking each as completed, then opens page five before leaving the application.
- **7. Automatic Reading Resume:** When Adam logs in again, StoryLand automatically opens page five, where he previously stopped. His completed pages remain saved, and opening page five has not automatically marked it as completed.
- **8. Student Progress Tracking:** Adam opens his personal dashboard and sees The Little Explorer marked as In Progress, with four of eight pages completed and a reading progress of 50%. He can also view his other assigned stories and their completion statuses.
- **9. Story Completion:** Adam continues reading and marks the remaining pages as completed. Once all eight pages are finished, StoryLand automatically updates the story status to Completed, with 100% progress.
- **10. Teacher Progress Monitoring:** The teacher opens the Young Readers class dashboard and selects The Little Explorer. The platform displays each student's completed pages, progress percentage, and reading status, allowing the teacher to identify students who have completed the story, are still reading, or have not started.

This scenario demonstrates the complete workflow of StoryLand, from class management and AI-assisted story creation to teacher approval, story assignment, individual reading, automatic resuming, and progress tracking for both students and teachers.

7. Evaluation Criteria

The project will be evaluated according to the following criteria:

Criterion	Description
Functional Completeness	Correct implementation of class management, story generation, publication, assignments, and reading progress.
Frontend and User Experience	Quality, responsiveness, accessibility, and child-friendly design of the Teacher and Student interfaces.
Backend and Database	Authentication, authorization, data management, and consistency of stories, assignments, and reading progress.
AI Integration	Relevance, coherence, age suitability, correct page counts, and reliability of generated stories.
Child Safety	Appropriate safeguards and mandatory teacher approval before publication.

Criterion	Description
Reading Progress	Accurate page completion, automatic resuming, correct percentages, and independent student progress.
Code Quality	Organization, readability, maintainability, and appropriate technical decisions.
Deployment	Accessibility and correct integration of the frontend, backend, database, and AI functionality.
Technical Understanding	Ability to explain the implemented workflows, AI integration, and important technical decisions.
Final Demonstration	Ability to demonstrate the complete application and its main features.

The expected outcome is a practical, engaging, and complete children's reading platform that demonstrates the ability to combine full-stack MERN development with AI-generated storytelling, classroom organization, and reliable reading progress tracking.